RAG System

How to Use — Quick Reference

FastAPI Backend server → **FAISS** Vector store → **MiniLM-L6** Embeddings → **Ollama llama3.1** LLM answers

Setup — run once

1 TERMINAL

`Install dependencies`

Install all required Python packages from requirements.txt.

```
pip install -r requirements.txt
```

2 TERMINAL

`Start Ollama (separate terminal)`

Pull and serve the llama3.1 model before starting the API.

```
ollama pull llama3.1
ollama serve
```

3 TERMINAL

`Start the FastAPI server`

Run on port 8000. Use --reload for development (auto-restarts on file change).

```
uvicorn rag_system:app --host 0.0.0.0 --port 8000 --reload
```

4 GET

`/health`

Verify the server is up, embedding model is loaded, and Ollama is reachable.

```
curl http://localhost:8000/health
```

Step 1 — Upload your factory files

5 POST

`/upload`

Upload one or more data files. Supported: CSV, PDF, TXT, MD, HTML. Files saved to rag_data/uploads/

```
curl -X POST http://localhost:8000/upload \
-F "files=@factory_production.csv" \
-F "files=@machine_logs.csv"
```

6 GET

`/files`

List all uploaded files with their names, sizes, and extensions.

```
curl http://localhost:8000/files
```

Step 2 — Build the FAISS index

7 POST

/index

Runs the full indexing pipeline: load docs → split into 500-char chunks → embed with MiniLM → store in FAISS. Run once. Re-run only when new files are added.

```
curl -X POST http://localhost:8000/index
```

8 GET

/status

Check if the index is ready. Returns state, total_chunks, files indexed, and build timestamp.

```
curl http://localhost:8000/status
```

Step 3 — Ask questions (RAG query)

9 POST

/query

Full RAG pipeline: embed question → FAISS cosine search → build augmented prompt → Ollama llama3.1 → returns answer + sources used.

```
curl -X POST http://localhost:8000/query \
-H "Content-Type: application/json" \
-d '{"question": "Which machine had most downtime?",
"top_k": 5, "temperature": 0.0}'
```

Response fields

answer — the LLM's grounded answer text
sources — list of chunks used (text, source file, score)
model — ollama model name used
latency_ms — total response time in ms

Python usage (call from code)

```
import requests # Upload with open('factory.csv', 'rb') as f:
requests.post('http://localhost:8000/upload', files={'files': f}) # Build index
requests.post('http://localhost:8000/index') # Query resp =
requests.post('http://localhost:8000/query', json={'question': 'Which machine had most
downtime?', 'top_k': 5}) print(resp.json()['answer']) print(resp.json()['sources']) # shows
which rows were used
```

Management routes

Route	Method	What it does
/index	DELETE	Wipe FAISS index. Uploaded files stay. Re-run POST /index to rebuild.
/files/{name}	DELETE	Delete a specific uploaded file. Re-index after.
/docs	GET	Interactive Swagger UI — test all routes from your browser.

Important: Always use the same embedding model for both indexing and querying. The default is `all-MiniLM-L6-v2`. Changing the model after indexing will break retrieval — you must delete and rebuild the index.